SLIIT UNI
THE KNOWLEDGE UNIVERSITY

IT4010 Research Project – 2025 July

Topic Assessment Form

Project ID : 25-26J-524

1. Topic (12 words max)

AI-Enhanced Self-Healing RPA Framework: A Distributed Microservices Architecture for Autonomous Bot Recovery and Intelligent Code Generation

2. Research group the project belongs to

CoEAI - Centre of Excellence for AI

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	25-26J-524
Year	4

5. Brief description of the research problem including references (200 – 500 words max) – references not included in word count.

Robotic Process Automation (RPA) is a technology that helps businesses automate boring, repetitive tasks. Companies use RPA bots to click buttons, fill forms, and move data between different software applications, just like a human would do. The RPA market is growing fast and many companies are using it to save time and money.

However, RPA has a big problem: when websites or software applications change their design or layout, the RPA bots stop working. This happens because RPA bots rely on finding specific buttons, text boxes, and other elements on the screen. When these elements move or change, the bot cannot find them anymore and breaks down.

For example, if a company's website updates and moves the "Submit" button to a different location, the RPA bot will keep looking for the button in the old location and fail to complete its task. Research shows that fixing these broken bots takes about 15 minutes each time, and companies spend up to 30% of their RPA maintenance time just fixing these problems.

Currently, when RPA bots break, a human programmer must manually fix the code. This creates several problems: it costs money to hire people to fix the bots, the automated processes stop working until someone fixes them, and companies lose trust in RPA technology because it breaks too often.

Some RPA companies have tried to create "self-healing" solutions, but these are very basic. They only fix simple problems and don't use smart technology like artificial intelligence. Most importantly, they don't work well when companies have many bots running on different computers.

This creates a big gap between what RPA promises (fully automated processes) and what actually happens (lots of manual work to keep bots running). There is no complete solution that can automatically detect problems, fix broken code, and work across many different systems.

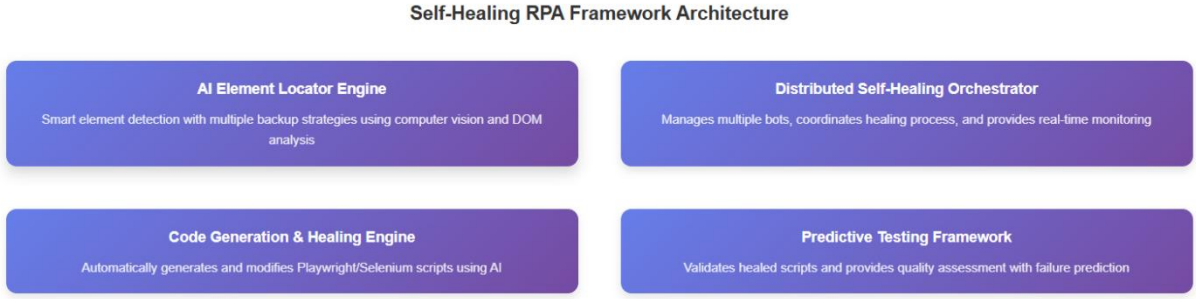
Our research aims to solve this problem by creating a smart RPA system that can heal itself when things go wrong. We will use artificial intelligence to predict when bots might break and automatically generate new code to fix them. Our system will work on multiple computers at the same time and will include smart testing to make sure the fixes actually work.

This research will help make RPA more reliable and reduce the need for human intervention, making automation truly automatic.

References

1. A Study on the Implementation of Robotic Process Automation (RPA) to Decrease the Time Required for the Documentation Process. Retrieved from https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9786521
2. Robotic Process Automation (RPA): A software bot for healthcare sector Retrieved from https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10090996
3. Deloitte Insights. (2023). Robotic process automation (RPA): Scaling intelligent automation. Retrieved from https://www2.deloitte.com/us/en/insights/focus/technology-and-the-future-ofwork/intelligent-automation-2022-survey-results.html
4. Next Generation Automation. (2019). How Self Healing Automation works? Retrieved from https://www.nextgenerationautomation.com/post/how-self-healingautomation-works
5. ACCELQ. (2025). Self-Healing Test Automation: Reduce Failures & Boost Efficiency. Retrieved from https://www.accelq.com/blog/self-healing-test-automation/

6. Brief description of the nature of the solution including a conceptual diagram (250 words max)



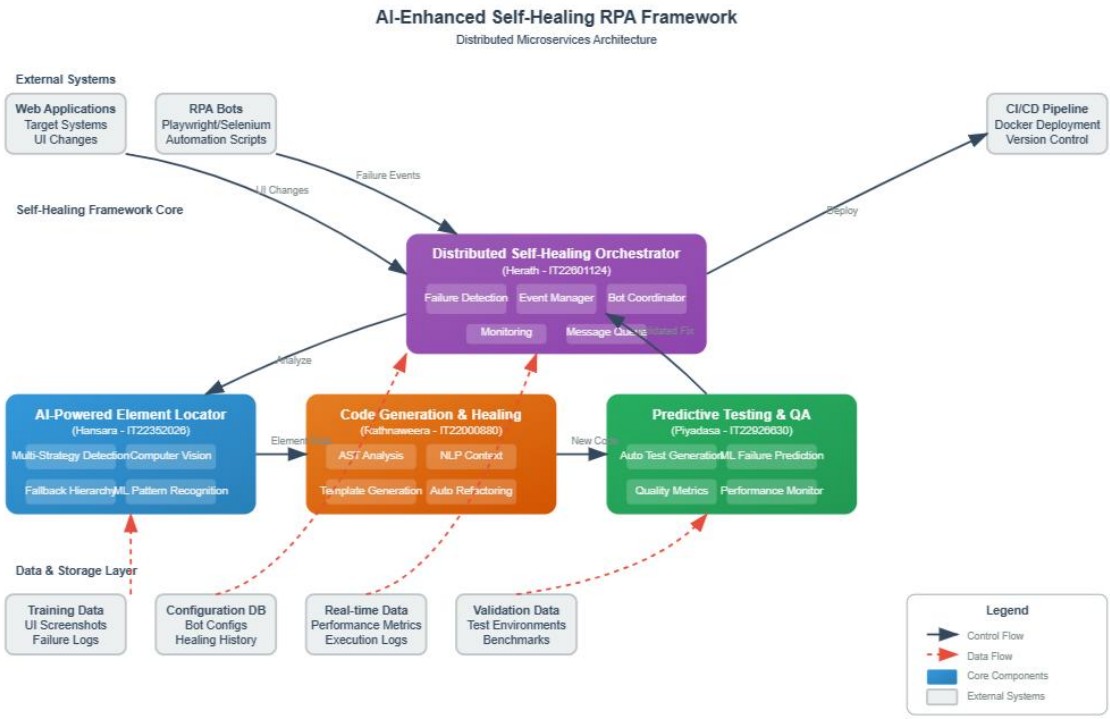
Our intelligent self-healing RPA framework automatically detects and fixes broken automation scripts without human help. When a website or application changes, the system uses artificial intelligence to find new ways to interact with the updated interface and generates new code to replace the broken parts.

The solution works through four main components working together: an AI-powered element finder that locates buttons and forms even when they move, a distributed system manager that coordinates multiple bots across different computers, an intelligent code generator that writes new automation scripts automatically, and a smart testing system that checks if the fixes work properly.

When a bot breaks, the system immediately detects the failure, analyzes what changed in the application, generates new code to handle the changes, tests the new code to ensure it works, and deploys the fixed bot back into production. This entire process happens automatically within minutes, compared to the current manual approach that takes hours or days.

The framework is built using modern technologies like Docker containers for easy deployment, message queues for communication between components, and machine learning algorithms for intelligent decision-making. This makes the system scalable, reliable, and capable of handling multiple automation processes simultaneously across different environments.

High Level Diagram:



7. Brief description of specialized domain expertise, knowledge, and data requirements (300 words max)

Specialized Domain Expertise

Robotic Process Automation (RPA): Deep understanding of RPA tools like Playwright and Selenium, including element locators (XPath, CSS selectors), web automation patterns, and common failure scenarios. Knowledge of RPA lifecycle management, bot deployment strategies, and maintenance challenges in enterprise environments.

Artificial Intelligence & Machine Learning: Expertise in computer vision for UI element recognition, natural language processing for understanding application contexts, and machine learning algorithms for pattern recognition and predictive analysis. Understanding of model training, evaluation metrics, and deployment in production systems.

Distributed Systems Architecture: Knowledge of microservices design patterns, containerization with Docker, message queue systems (Redis/RabbitMQ), and API

development. Understanding of scalability, fault tolerance, and inter-service communication in distributed environments.

Software Engineering & DevOps: Proficiency in automated testing frameworks, CI/CD pipeline design, version control systems, and code generation techniques using Abstract Syntax Trees (AST). Knowledge of software quality metrics and automated deployment strategies.

Knowledge Requirements

Understanding of web technologies (HTML, CSS, JavaScript DOM manipulation), software testing methodologies, database systems for storing bot configurations and healing history, and cloud deployment platforms. Familiarity with performance monitoring tools and system observability practices.

Data Requirements

Training Datasets: Large collections of web application screenshots and HTML structures before/after UI changes to train AI models for element recognition. Historical RPA failure logs with corresponding UI changes to understand common failure patterns.

Configuration Data: Element locator patterns, application-specific rules, and bot behavior configurations. Success/failure metrics from existing RPA deployments to establish baseline performance measurements.

Real-time Data: Live application monitoring data, system performance metrics, and user interaction patterns. Bot execution logs, error traces, and healing attempt results for continuous learning and improvement.

Validation Data: Test application environments with controlled UI changes, benchmark datasets for comparing healing effectiveness, and performance metrics from similar self-healing automation tools for competitive analysis.

The project requires access to enterprise RPA environments for realistic testing scenarios and collaboration with organizations currently using RPA to validate practical applicability and gather real-world feedback on healing effectiveness.

8. Objectives and Novelty

Main Objective:
To develop an intelligent self-healing RPA framework that automatically detects, analyzes, and repairs broken automation scripts caused by user interface changes, reducing manual maintenance overhead and improving RPA system reliability through AI-driven predictive analysis and automated code generation.

Member Name with Registration No	Sub Objective	Tasks	Novelty
Rathnaweera TT (IT22000880)	Create Intelligent Code Generation & Healing Engine for automated script recovery and modification	• Develop AST-based code analysis and modification • Implement template-based code generation system • Create NLP-powered context understanding • Build automated script refactoring capabilities • Integrate version control for healing history	**First AI-powered automatic code generation** for RPA healing. **Novel use of AST manipulation** combined with NLP for intelligent script modification.
Hansara K D K U (IT22352026)	Develop AI-Powered Element Locator Engine for intelligent UI	• Implement multi-strategy element detection (XPath, CSS, visual)	**Novel AI-driven approach** combining DOM analysis with computer vision for robust element detection. **First implementation** of multi-strategy locator scoring in RPA context.

	element detection and recognition	• Develop computer vision algorithms for UI element recognition • Create fallback hierarchy system for failed locators • Build element reliability scoring mechanism • Integrate machine learning for pattern recognition	
Piyadasa J D C T (IT22926630)	Implement Predictive Testing & Quality Assessment Framework for healing validation and performance optimization	• Design automated test generation for healed scripts • Develop ML-based failure prediction models • Create comprehensive quality metrics system • Implement CI/CD integration with quality gates • Build performance regression detection	**First predictive quality assessment** system for RPA healing effectiveness. **Novel integration** of ML-based failure prediction with automated testing in RPA context.
Herath H M T W (IT22601124)	Design Distributed Self-Healing Orchestrator for scalable multi-bot management and coordination	• Architect microservices-based healing system • Implement real-time failure detection mechanisms • Develop distributed bot coordination protocols • Create centralized monitoring dashboard	**First distributed architecture** specifically designed for RPA self-healing. **Novel microservices approach** to RPA resilience with real-time orchestration.

		• Build event-driven healing trigger system	

9. Individual component description of how it is complied with the specialization.

Member Name with Registration No	Description
Rathnaweera TT (IT22000880)	SE Domain: Automated Code Generation & Software Transformation Implements Abstract Syntax Tree (AST) manipulation, Template Method Pattern for code generation, and Visitor Pattern for code analysis. Applies compiler design principles and software transformation techniques. SE Practices Applied: • Code Analysis:- Static code analysis to understand existing automation scripts • Template Engineering:- Reusable code templates for common automation patterns • Version Control Integration:- Git-based tracking of code generation and modifications • Code Quality Metrics:- Automated assessment of generated code quality • Refactoring Techniques:- Automated code improvement and optimization
Hansara K D K U (IT22352026)	SE Domain: Software Architecture & Design Patterns Implements Strategy Pattern for multiple element detection approaches, Factory Pattern for locator creation, and Observer Pattern for element state monitoring. Applies architectural principles like separation of concerns and modularity. SE Practices Applied: • Code Reusability:- Modular locator strategies that can be reused across different applications • Error Handling:- Robust exception handling for failed element detection attempts • Performance Optimization:- Efficient algorithms for element matching and scoring • API Design:- Clean interfaces for locator engine integration with other components • Unit Testing:- Comprehensive test suites for each locator strategy

Piyadasa J D C T (IT22926630)	SE Domain: Predictive Testing & Quality Assessment Implements Test Automation Framework design, Page Object Model for maintainable tests, and Continuous Integration/Continuous Deployment (CI/CD) practices. Applies testing methodologies and quality metrics. SE Practices Applied: • Test-Driven Development:- Automated test generation for healed components • CI/CD Pipeline Integration:- Automated testing in deployment workflows • Quality Gates:- Automated quality checkpoints before deployment • Regression Testing:- Automated detection of performance and functionality regressions • Metrics Collection:- Comprehensive quality and performance metrics gathering
Herath H M T W (IT22601124)	SE Domain: Distributed Systems & Microservices Architecture Applies Microservices Architecture principles, implements Service Discovery, Load Balancing, and Circuit Breaker Pattern. Focuses on scalability, fault tolerance, and inter-service communication design. SE Practices Applied: • Containerization:- Docker-based deployment for consistent environments • API Gateway Pattern:- Centralized entry point for service communication • Event-Driven Architecture:- Asynchronous message handling for system events • Monitoring & Logging:- Distributed logging and real-time system monitoring • Configuration Management:- Environment-specific configurations and secrets management

10. Supervisor details

	Title	First Name	Last Name	Signature
Supervisor	Prof.	Nuwan	Kodagoda	
Co-Supervisor	Prof.	Dharshana	Kasthurirathne	
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				

<u>This part is to be filled by the Topic Screening Staff members.</u>

a) Does the chosen research topic possess a comprehensive scope suitable for a final-year project?

Yes		No	

b) Does the proposed topic exhibit novelty?

Yes		No	

c) Do you believe they have the capability to successfully execute the proposed project?

Yes		No	

d) Do the proposed sub-objectives reflect the students' areas of specialization?

Yes		No	

e) Supervisor's Evaluation and Recommendation for the Research topic:

IT4010 Research Project – 2025 July

Topic Assessment Form

Acceptable: Mark/Select as necessary

Topic Assessment Accepted	
Topic Assessment Accepted with minor changes*	
Topic Assessment to be Resubmitted with major changes*	
Topic Assessment Rejected. Topic must be changed	

* Detailed comments given below

Comments

Staff Member's Name	Signature

*__Important__:

1. According to the comments given by the evaluator, make the necessary modifications and get the approval by the **Evaluator**.
2. If the project topic is rejected, identify a new topic, and request the RP Team for a new topic assessment.